Graduation Project System Architecture Specification

AI-Powered Adaptive Learning Platform: Requirements, Multi-Agent Framework, and Evaluation Models

> **Project Implementation Objective & Scope**
>
> This document serves as the formal comprehensive system design specifications. The scope includes revising and mapping tables/FK relationships for the relational SQL database ('GraduationProject'), structuring a high-density Qdrant Vector Database instance, and deploying a decoupled multi-agent architecture responsible for real-time user profiling, conceptual evaluation, and predictive study pathway generation.

1. Core System Requirements & Backlog Tracker

- **Topic Extraction & Tracking:** Dynamically extract studied concepts from video assets or blog platforms to explicitly monitor user learning pathways.
- **Gated Evaluation Checking:** Administer granular checkpoints/exams based on current topics, enforcing an application-level session lock until complete.
- **User Profile Optimization:** Continuous telemetry update cycles, modifying 'UserTopicMastery', progression metrics, and depth flags using empirical evaluation outcomes.
- **Knowledge Graph Mapping:** Establish directed, tree-like relational structures across topics under cohesive domains (e.g., nesting 'Binary Search' and 'Dynamic Programming' under 'Algorithms').
- **High-Level Mastery Aggregation:** Render holistic domain metrics (e.g., scoring general 'Machine Learning' or 'Statistics' readiness via weighted averages of underlying sub-topic scores).
- **Predictive Recommendation Pipeline:** Generate automated study path recommendations targeting explicit weaknesses or prerequisites based on accumulated user matrices.

- **Autonomous Career Synthesis (Optional):** Deploy an isolated agent module to parse user performance records and dynamically generate clean professional CV templates and LinkedIn summaries.

2. Decoupled Multi-Agent Architecture

The cognitive engine consists of specialized LLM-backed autonomous units interacting through structured pipelines:

2.1 Profile Agent

Responsible for background tracking and profile data synchronization. It translates raw user interaction signals into formal system evidence metrics.

- **Micro-tasks:** Topic Extraction, Mastery Estimation, Weakness Detection, and State Management.
- **Monitored State Fields:** UserTopicMastery, Evidence Vector, User Interest, Contextual Confidence.

2.2 Recommendation Agent

Calculates personalized content strategies and linear next-step curricula by applying graph traversal logic across structural constraints.

Ingested Inputs	Generated Outputs
• Explicit Long-term Goals • Multidimensional Mastery Matrix	• Target Topics ('What to study next') • Curated Learning Resource Recommendations
• Global Knowledge Graph Relationships • Isolated Weaknesses & Study History	• Optimized Learning Pathways • Contextual Topic Groupings

2.3 Assistant Agent

Handles runtime chat operations, custom user prompts, memory querying, and concept explanations. To enforce strong systemic isolation boundaries, this agent possesses zero database mutation or direct modification rights.

2.4 Career Agent (Optional)

Aggregates student project logs, historical mastery tiers, career targets, and certifications to output customized CV drafts, LinkedIn profiles, market advice, and automated skill-gap reports.

3. Operational Telemetry Pipeline

System state tracking executes sequentially as a structured transaction process:

Step 1: Session Log: User finishes consumer task; a new 'StudySession' row is added to the relational store.

Step 2: Vector Push: System creates a semantic 'Session Summary' narrative, transforms it, and persists it inside Qdrant.SessionEmbeddings.

Step 3: Concept Extraction: Profile Agent reviews text to parse atomic leaf nodes (e.g., isolating 'PDF', 'PMF', and 'Normal Distribution').

Step 4: Evidence Harvesting: Data ingestion from multiple validation vectors: Study Time, Quiz Scores, User Questions, Retention Tests, and Projects.

Step 5: Metric Normalization: The multi-channel signals are mathematically compiled into a bounded Evidence Score (0.00 to 1.00).

Step 6: Mastery Update: The system calculates the updated mastery metrics via an exponential moving average (EMA) smoothing algorithm.

Step 7: Confidence Bump: Increments tracking telemetry registers natively: Confidence += evidence_count.

Step 8: Global Node Recalculation: Triggers recursive upward averaging across parent tree branches (e.g., recomputing holistic 'Statistics' readiness).

4. Mathematical Evaluation Framework

4.1 Mastery Scaling (Exponential Moving Average)

$$\text{NewMastery} = (\text{OldMastery} \times (1 - \alpha)) + (\text{EvidenceScore} \times \alpha)$$

Where the learning velocity coefficient is statically locked at $\alpha = 0.20$. This prevents rapid, unstable spikes in user profile data. For instance, updating an OldMastery of 0.30 with an EvidenceScore of 0.74 computes as: (0.30 × 0.8) + (0.74 × 0.2) = 0.388, yielding an updated permanent status profile of 0.39.

4.2 Evidence Score Multi-Channel Composition

Signal Stream	Weight	Algorithmic Composition Rules
Quiz / Assessment	40%	Linear accuracy fraction: correct_answers /

		total_questions. (e.g., 8/10 = 0.80)
Question Quality	20%	LLM scores cognitive depth of questions (Low-level: 0.3, Understanding: 0.7, Reasoning: 0.9)
Study Completion	15%	Bounded consumption ratio tracker: min(duration_watched / expected_duration, 1.0)
Retention Test	15%	Time-delayed assessment checks to explicitly model long-term cognitive decay graphs.
Projects / Exercises	10%	Practical execution verification (e.g., building a functional portfolio project spans 0.80 - 1.00)

4.3 Multivariable Confidence Score Structure

$$\text{Confidence} = 0.30 \times \text{SessionScore} + 0.30 \times \text{AssessmentScore} + 0.20 \times \text{RecencyScore} + 0.20 \times \text{ConsistencyScore}$$

- **SessionScore:** Interaction frequency normalization matrix: min(sessions / 20, 1.0).
- **AssessmentScore:** Evaluation frequency normalization matrix: min(assessments / 10, 1.0).
- **RecencyScore:** Models learning degradation using an exponential function: exp(-DaysSinceStudy / 60).
- **ConsistencyScore:** Evaluates stability across historical testing logs using normalized deviation: 1.0 - normalized_std_dev.

5. Comprehensive Database Architecture

The system relies on a dual-engine architecture: SQL Server manages structured relationships and analytical states, while Qdrant provides high-speed vector storage for semantic memory blocks.

5.1 Relational Architecture (SQL Server)

This represents the transactional system of record. Every entity enforces strict relational dependencies, cascade parameters, and primary tracking indexes:

Table: Users

Column	Type	Constraints / Reference
UserId	BIGINT	PRIMARY KEY, IDENTITY(1,1)
Name	NVARCHAR(150)	NOT NULL
Email	VARCHAR(255)	NOT NULL, UNIQUE
CreatedAt	DATETIME2	NOT NULL, DEFAULT GETUTCDATE()

Table: Goals

Column	Type	Constraints / Reference
GoalId	BIGINT	PRIMARY KEY, IDENTITY(1,1)
UserId	BIGINT	FOREIGN KEY REFERENCES Users(UserId)
Title	NVARCHAR(200)	NOT NULL
Priority	INT	NOT NULL DEFAULT 1
CreatedAt	DATETIME2	NOT NULL, DEFAULT GETUTCDATE()

Examples / Context: Become ML Engineer | Become Backend Developer | Learn Data Science

Table: Topics

Column	Type	Constraints / Reference
TopicId	INT	PRIMARY KEY, IDENTITY(1,1)
Name	NVARCHAR(150)	NOT NULL, UNIQUE
Description	NVARCHAR(MAX)	NULL
Type	VARCHAR(50)	NOT NULL (Domain, Concept, Technique, Tool, Career)
Difficulty	INT	NOT NULL DEFAULT 1
EstimatedHours	DECIMAL(5,2)	NOT NULL

Examples / Context: Machine Learning (Domain) | Probability Distribution (Concept) | Random Forest (Technique)

Table: TopicRelationships

Column	Type	Constraints / Reference
RelationshipId	INT	PRIMARY KEY, IDENTITY(1,1)
SourceTopicId	INT	FOREIGN KEY REFERENCES Topics(TopicId)
TargetTopicId	INT	FOREIGN KEY REFERENCES Topics(TopicId)
RelationshipType	VARCHAR(50)	NOT NULL (contains, prerequisite_for, required_for, related_to,

		etc.)
Weight	DECIMAL(3,2)	NOT NULL DEFAULT 1.00

Examples / Context: Statistics -> contains -> Probability Distribution | Probability Distribution -> prerequisite_for -> Bayes Theorem

Table: UserTopicMastery

Column	Type	Constraints / Reference
UserId	BIGINT	FOREIGN KEY REFERENCES Users(UserId)
TopicId	INT	FOREIGN KEY REFERENCES Topics(TopicId)
Mastery	DECIMAL(3,2)	NOT NULL DEFAULT 0.00 (Scale 0-1)
Confidence	DECIMAL(3,2)	NOT NULL DEFAULT 0.00 (Scale 0-1)
Interest	DECIMAL(3,2)	NOT NULL DEFAULT 0.50 (Scale 0-1)
EvidenceCount	INT	NOT NULL DEFAULT 0
LastUpdated	DATETIME2	NOT NULL, DEFAULT GETUTCDATE()

Examples / Context: Composite Key: (UserId, TopicId)

Table: Resources

Column	Type	Constraints / Reference
ResourceId	BIGINT	PRIMARY KEY, IDENTITY(1,1)

Title	NVARCHAR(255)	NOT NULL
Type	VARCHAR(50)	NOT NULL (Youtube, Article, PDF, Course, Book, Documentation)
Url	VARCHAR(2048)	NOT NULL
Difficulty	INT	NOT NULL DEFAULT 1
Depth	INT	NOT NULL DEFAULT 1
EstimatedMinutes	INT	NOT NULL
CreatedAt	DATETIME2	NOT NULL, DEFAULT GETUTCDATE()

Table: ResourceTopicCoverage

Column	Type	Constraints / Reference
ResourceId	BIGINT	FOREIGN KEY REFERENCES Resources(ResourceId)
TopicId	INT	FOREIGN KEY REFERENCES Topics(TopicId)
CoverageWeight	DECIMAL(3,2)	NOT NULL DEFAULT 1.00
DifficultyContribution	DECIMAL(3,2)	NOT NULL DEFAULT 1.00

Examples / Context: Composite Key: (ResourceId, TopicId)

Table: StudySessions

Column	Type	Constraints / Reference
SessionId	BIGINT	PRIMARY KEY, IDENTITY(1,1)

UserId	BIGINT	FOREIGN KEY REFERENCES Users(UserId)
ResourceId	BIGINT	FOREIGN KEY REFERENCES Resources(ResourceId)
StartedAt	DATETIME2	NOT NULL
EndedAt	DATETIME2	NULL
DurationMinutes	INT	AS DATEDIFF(minute, StartedAt, EndedAt)
SessionSummary	NVARCHAR(MAX)	NULL

Table: Evidence

Column	Type	Constraints / Reference
EvidenceId	BIGINT	PRIMARY KEY, IDENTITY(1,1)
SessionId	BIGINT	FOREIGN KEY REFERENCES StudySessions(SessionId)
TopicId	INT	FOREIGN KEY REFERENCES Topics(TopicId)
Type	VARCHAR(50)	NOT NULL (study_time, quiz, question, project, assessment, certificate)
Score	DECIMAL(3,2)	NOT NULL
CreatedAt	DATETIME2	NOT NULL, DEFAULT GETUTCDATE()

Table: Questions

Column	Type	Constraints / Reference
QuestionId	BIGINT	PRIMARY KEY, IDENTITY(1,1)
UserId	BIGINT	FOREIGN KEY REFERENCES Users(UserId)
SessionId	BIGINT	FOREIGN KEY REFERENCES StudySessions(SessionId)
QuestionText	NVARCHAR(MAX)	NOT NULL
CreatedAt	DATETIME2	NOT NULL, DEFAULT GETUTCDATE()

Table: Projects

Column	Type	Constraints / Reference
ProjectId	BIGINT	PRIMARY KEY, IDENTITY(1,1)
UserId	BIGINT	FOREIGN KEY REFERENCES Users(UserId)
Title	NVARCHAR(200)	NOT NULL
Description	NVARCHAR(MAX)	NULL
StartDate	DATETIME2	NOT NULL
EndDate	DATETIME2	NULL

Table: Certificates

Column	Type	Constraints / Reference
CertificateId	BIGINT	PRIMARY KEY, IDENTITY(1,1)
UserId	BIGINT	FOREIGN KEY REFERENCES Users(UserId)
Title	NVARCHAR(200)	NOT NULL
Issuer	NVARCHAR(150)	NOT NULL
IssueDate	DATETIME2	NOT NULL
Url	VARCHAR(2048)	NULL

Table: UserDomains

Column	Type	Constraints / Reference
UserId	BIGINT	FOREIGN KEY REFERENCES Users(UserId)
TopicId	INT	FOREIGN KEY REFERENCES Topics(TopicId)
Score	DECIMAL(3,2)	NOT NULL

Examples / Context: Composite Key: (UserId, TopicId) - Aggregated domain caching layer

5.2 Semantic Memory Architecture (Qdrant)

Qdrant stores unstructured vector indices to perform semantic retrieval. It does not replace the SQL backend; instead, it manages sparse or dense embeddings mapping back to core relational IDs via specific payload formats:

Collection: ResourceEmbeddings

```
{
   "id": "resource_{ResourceId}",
   "vector": [1536 dimensions],
   "payload": {
     "resource_id": 100,
     "topics": ["Probability Distribution", "Normal Distribution"]
   }
 }
```

Collection: SessionEmbeddings

```
{
   "id": "session_{SessionId}",
   "vector": [1536 dimensions],
   "payload": {
     "user_id": 1,
     "topics": ["Probability Distribution"]
   }
 }
```

Collection: QuestionEmbeddings

```
{
   "id": "question_{QuestionId}",
   "vector": [1536 dimensions],
   "payload": {
     "user_id": 1,
     "topics": ["Bayes Theorem"]
   }
 }
```

Collection: NoteEmbeddings

```
{
   "id": "note_{NoteId}",
   "vector": [1536 dimensions],
   "payload": {
     "user_id": 1
   }
 }
```

Collection: ProjectEmbeddings

```
{
   "id": "project_{ProjectId}",
   "vector": [1536 dimensions],
   "payload": {
     "user_id": 1
   }
 }
```

6. Algorithmic Content Recommendation Logic

$$\text{Priority} = (\text{RequiredMastery} - \text{Mastery}) \times \text{Confidence} \times \text{Interest} \times \text{CareerRelevance}$$

Execution Strategy Example Walkthrough:

```
Target Career Goal: Machine Learning Engineer
Active Knowledge Node Metrics:
 Statistics Core
   ├── Probability Distribution  -> Mastery: 0.65
   ├── Bayes Theorem             -> Mastery: 0.20 | Confidence: 0.30
   └── Hypothesis Testing        -> Mastery: 0.15 | Confidence: 0.85
```

Evaluation Output: The Recommendation Agent prioritizes Hypothesis Testing over Bayes Theorem. Even though both present sharp baseline mastery deficits, Hypothesis Testing has a much higher Confidence metric (0.85 vs 0.30), indicating the system possesses robust evaluation history proving an explicit weakness, rather than a simple lack of operational tracking data.